Entrega — Unidad de Control y ALU

La instrucción decide, la ALU calcula

Integrantes: Marcos López | Luis Angel Ramirez **Carnés:** 231928 | 221165

Machote de entrega · Parte del procesador MIPS: Unidad de Control y ALU

Entregan dos ejercicios (Unidad de Control y ALU). Cada uno pide código Verilog, testbench comentado, captura de consola, captura de GTKWave y detalle de los casos de prueba.

Antes de empezar

Qué se entrega

Los nombres de los módulos, de los puertos y de los archivos los escoge cada grupo.

Módulo	Puertos	Qué hace
`Unidad de Control`	Entrada: opcode (6 bits). Salidas: RegDst, ALUSrc, RegWrite, Jump (1 bit cada una) y ALUOp (2 bits), todas reg.	Decodifica el opcode de la instrucción y genera las señales que controlan el resto del procesador. Es lógica combinacional.
`ALU`	Entradas: dos operandos (32 bits) y control (4 bits). Salidas: resultado (32 bits, reg) y bandera de cero.	Suma, resta, multiplicación y división. La bandera de cero vale 1 cuando el resultado es 0.

Señales que debe generar la Unidad de Control

«—» significa que el enunciado no fija ese valor: decídanlo ustedes y documéntenlo.

Tipo	RegDst	ALUSrc	RegWrite	Jump	ALUOp
R	1	0	1	—	10
I (ej. MOVI)	0	1	1	—	00
J (ej. JMP)	0	0	0	1	00

Formato de salida sugerido

```
opcode=000000 -> RegDst=1 ALUSrc=0 RegWrite=1 Jump=0 ALUOp=10
ctl=0001 src1=50 src2=15 -> result=35 (0x00000023) zero=0
```

Detalles técnicos que evitan sorpresas

- **Valores por defecto.** En un always @(*) asigna un valor a TODAS las salidas antes del case. Si no, las señales que una rama no toca quedan en latch o en x (por ejemplo Jump en R e I, o RegWrite en J).
- **Con signo o sin signo.** Los vectores de Verilog son sin signo. Con 5 − 10 el resultado es 0xFFFFFFFB y %d lo imprime como 4294967291; usa $signed() para mostrarlo como −5. En la división, −20 / 4 sin signo da 1073741819: para dividir con signo, $signed(src1) / $signed(src2).
- **División entre cero.** En simulación da x (y la bandera de cero también). Decide qué debe devolver tu ALU y documéntalo.
- **Multiplicación.** El producto de dos operandos de 32 bits se trunca a 32 bits: 65536 × 65536 da 0 y la bandera de cero sube.
- **Ondas y comandos.** Cada testbench lleva $dumpfile("nombre.vcd") y $dumpvars(0, nombre_del_tb). iverilog -o salida.vvp modulo.v testbench.v → vvp salida.vvp → gtkwave nombre.vcd

Ejercicio 1 — Unidad de Control

1. Código Verilog de la Unidad de Control

Decodifica el opcode y genera RegDst, ALUSrc, RegWrite, Jump y ALUOp. Comenta qué hace cada rama del case y por qué los valores por defecto están al inicio.

Tu código — la Unidad de Control

```
`timescale 1ns/1ns

module ControlUnit (
    input wire [5:0] opcode,
    output reg RegDst,
    output reg ALUSrc,
    output reg RegWrite,
    output reg Jump,
    output reg [1:0] ALUOp
);

    // Codificacion elegida para este procesador educativo.
    localparam [5:0] OP_R        = 6'b000000;
    localparam [5:0] OP_MOVI     = 6'b001000;
    localparam [5:0] OP_MOVI_ALT = 6'b001001;
    localparam [5:0] OP_JMP      = 6'b000010;

    // Decodificacion combinacional.
    always @(*) begin

        // Valores por defecto para todas las salidas.
        // Evitan latches y deshabilitan escritura y salto
        // cuando el opcode no esta definido.
        RegDst   = 1'b0;
        ALUSrc   = 1'b0;
        RegWrite = 1'b0;
        Jump     = 1'b0;
        ALUOp    = 2'b00;

        case (opcode)

            // Tipo R: dos operandos de registros.
            // ALUControl utiliza funct para elegir la operacion.
            OP_R: begin
                RegDst   = 1'b1;
                ALUSrc   = 1'b0;
                RegWrite = 1'b1;
                ALUOp    = 2'b10;
            end

            // MOVI: el segundo operando es un inmediato.
            OP_MOVI: begin
```

```
                RegDst   = 1'b0;
                ALUSrc   = 1'b1;
                RegWrite = 1'b1;
                ALUOp    = 2'b00;
            end

            // Segunda codificacion de MOVI.
            // Tiene exactamente el mismo comportamiento.
            OP_MOVI_ALT: begin
                RegDst   = 1'b0;
                ALUSrc   = 1'b1;
                RegWrite = 1'b1;
                ALUOp    = 2'b00;
            end

            // Salto incondicional: no escribe registros.
            OP_JMP: begin
                Jump = 1'b1;
            end

            // Conservar los valores por defecto.
            default: begin
            end

        endcase
    end

endmodule
```

La Unidad de Control decodifica el opcode mediante un bloque combinacional always @(*) y una estructura case. Al inicio del bloque se asignan valores por defecto a todas las salidas para evitar latches y asegurar que ninguna señal conserve el valor de una instrucción anterior.

Para las instrucciones tipo R, se activa RegDst para seleccionar el campo rd como destino, se mantiene ALUSrc en cero para utilizar un segundo operando de registro y se habilita RegWrite. La salida ALUOp toma el valor 10, indicando que ALUControl debe utilizar el campo funct para seleccionar la operación.

Para MOVI y MOVI_ALT, RegDst vale cero, ALUSrc vale uno y RegWrite vale uno. ALUOp toma el valor 00, correspondiente a MOVI en nuestro ALUControl. MOVI_ALT se definió como una segunda codificación de MOVI, con el mismo comportamiento.

Para JMP, se activa Jump y se mantiene RegWrite en cero, evitando la escritura en el banco de registros. Los opcodes no definidos producen todas las señales en cero, por lo que no habilitan escritura ni salto.

2. Testbench de la Unidad de Control

Cambia el opcode en cada ciclo (R, I y J al menos) y muestra las señales generadas. Verifica los valores esperados, no solo los imprimas.

Tu testbench — la Unidad de Control

```
`timescale 1ns/1ns

module tb_ControlUnit;

    reg [5:0] opcode;

    wire RegDst;
    wire ALUSrc;
    wire RegWrite;
    wire Jump;
    wire [1:0] ALUOp;

    integer pruebas;
    integer errores;

    ControlUnit uut (
        .opcode(opcode),
        .RegDst(RegDst),
        .ALUSrc(ALUSrc),
        .RegWrite(RegWrite),
        .Jump(Jump),
        .ALUOp(ALUOp)
    );

    // Aplicar un opcode y verificar todas las salidas.
    task probar;
        input [5:0] op;
        input esperado_RegDst;
        input esperado_ALUSrc;
        input esperado_RegWrite;
        input esperado_Jump;
        input [1:0] esperado_ALUOp;

        begin
            opcode = op;
            #10;

            pruebas = pruebas + 1;

            $display(
                "opcode=%b -> RegDst=%b ALUSrc=%b RegWrite=%b Jump=%b ALUOp=%b",
                opcode, RegDst, ALUSrc, RegWrite, Jump, ALUOp
            );
```

```
            // Comparar las cinco señales como un vector de 6 bits.
            // !== tambien detecta valores desconocidos.
            if ({RegDst, ALUSrc, RegWrite, Jump, ALUOp} !==
                {esperado_RegDst, esperado_ALUSrc,
                 esperado_RegWrite, esperado_Jump,
                 esperado_ALUOp}) begin

                errores = errores + 1;

                $display(
                    "ERROR prueba %0d: esperado=%b %b %b %b %b",
                    pruebas,
                    esperado_RegDst, esperado_ALUSrc,
                    esperado_RegWrite, esperado_Jump,
                    esperado_ALUOp
                );
            end
        end
    endtask

    initial begin
        $dumpfile("ControlUnit.vcd");
        $dumpvars(0, tb_ControlUnit);

        opcode = 6'b111111;
        pruebas = 0;
        errores = 0;

        $display("=== Pruebas de ControlUnit ===");
        $display("Orden: RegDst ALUSrc RegWrite Jump ALUOp");

        // 1. Tipo R.
        probar(6'b000000, 1'b1, 1'b0, 1'b1, 1'b0, 2'b10);

        // 2. Tipo I: MOVI.
        probar(6'b001000, 1'b0, 1'b1, 1'b1, 1'b0, 2'b00);

        // 3. Tipo I: MOVI_ALT, alias definido por el grupo.
        probar(6'b001001, 1'b0, 1'b1, 1'b1, 1'b0, 2'b00);

        // 4. Tipo J: JMP.
        probar(6'b000010, 1'b0, 1'b0, 1'b0, 1'b1, 2'b00);

        // 5. Opcode no definido: sin escritura ni salto.
        probar(6'b111111, 1'b0, 1'b0, 1'b0, 1'b0, 2'b00);

        // 6. Secuencia R -> J -> I.
        $display("--- Secuencia R -> J -> I ---");
```

```
        probar(6'b000000, 1'b1, 1'b0, 1'b1, 1'b0, 2'b10);
        probar(6'b000010, 1'b0, 1'b0, 1'b0, 1'b1, 2'b00);
        probar(6'b001000, 1'b0, 1'b1, 1'b1, 1'b0, 2'b00);

        // 7. Pasar de MOVI a un opcode no definido.
        // RegWrite y ALUSrc deben regresar a cero.
        probar(6'b111111, 1'b0, 1'b0, 1'b0, 1'b0, 2'b00);

        $display(
            "=== Fin: pruebas=%0d, errores=%0d ===",
            pruebas, errores
        );

        $finish;
    end

endmodule
```

3. Captura de la consola

Debe verse el comando de compilación y ejecución, y una línea por opcode probado con todas las señales de control.

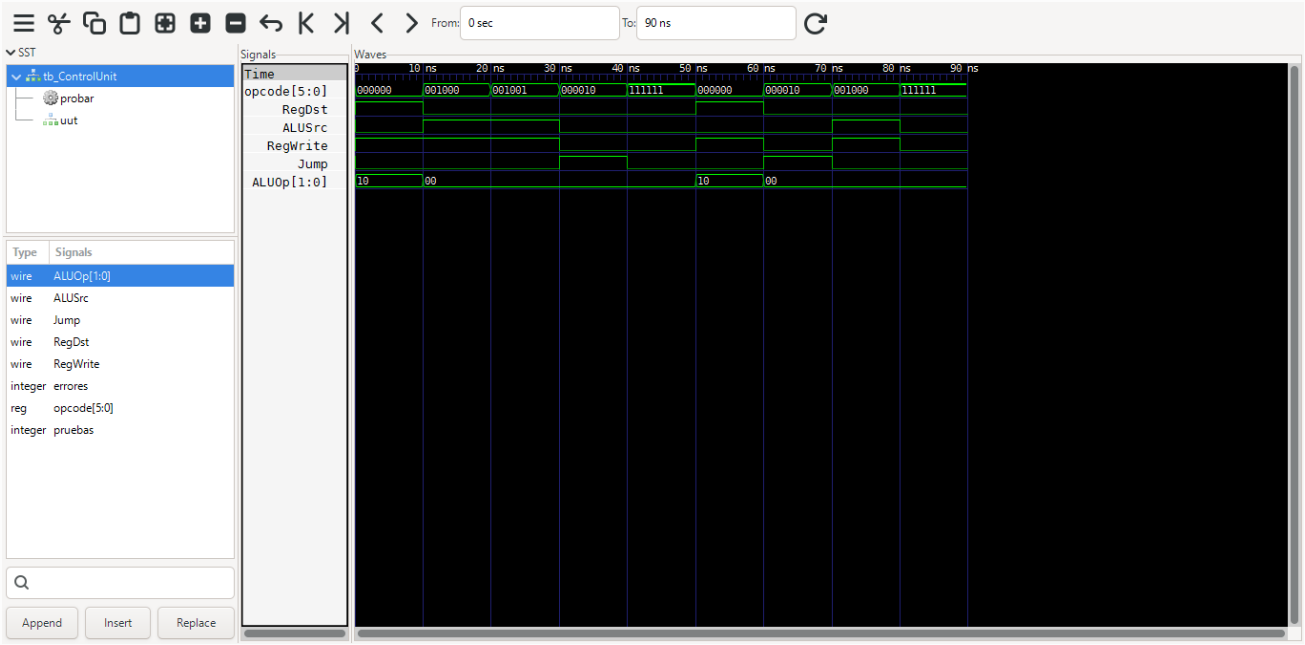
Captura — consola (Unidad de Control)

```
PS C:\Users\Usuario\Control_Registros> iverilog -s ControlUnit -o ControlUnit_check ControlUnit.v
PS C:\Users\Usuario\Control_Registros> iverilog -s tb_ControlUnit -o ControlUnit_sim ControlUnit.v tb_ControlUnit.v
PS C:\Users\Usuario\Control_Registros> vvp ControlUnit_sim | Tee-Object -FilePath ControlUnit_consola.txt
VCD info: dumpfile ControlUnit.vcd opened for output.
=== Pruebas de ControlUnit ===
Orden: RegDst ALUSrc RegWrite Jump ALUOp
opcode=000000 -> RegDst=1 ALUSrc=0 RegWrite=1 Jump=0 ALUOp=10
opcode=001000 -> RegDst=0 ALUSrc=1 RegWrite=1 Jump=0 ALUOp=00
opcode=001001 -> RegDst=0 ALUSrc=1 RegWrite=1 Jump=0 ALUOp=00
opcode=000010 -> RegDst=0 ALUSrc=0 RegWrite=0 Jump=1 ALUOp=00
opcode=111111 -> RegDst=0 ALUSrc=0 RegWrite=0 Jump=0 ALUOp=00
--- Secuencia R -> J -> I ---
opcode=000000 -> RegDst=1 ALUSrc=0 RegWrite=1 Jump=0 ALUOp=10
opcode=000010 -> RegDst=0 ALUSrc=0 RegWrite=0 Jump=1 ALUOp=00
opcode=001000 -> RegDst=0 ALUSrc=1 RegWrite=1 Jump=0 ALUOp=00
opcode=111111 -> RegDst=0 ALUSrc=0 RegWrite=0 Jump=0 ALUOp=00
=== Fin: pruebas=9, errores=0 ===
tb_ControlUnit.v:107: $finish called at 90 (1ns)
```

4. Captura de GTKWave

Señal de opcode y las cinco señales de control. Pon el opcode en binario y las señales como bit (o bus de 2 bits para ALUOp). Todos los opcodes probados a la vista (View → Zoom Fit).

Captura — GTKWave (Unidad de Control)



5. Detalle de los casos de prueba

Escribe el opcode que usó tu grupo en cada caso y las señales que esperas a mano (según el enunciado) antes de simular. Luego copia lo que dice la consola. Orden de las señales: RegDst · ALUSrc · RegWrite · Jump · ALUOp, por ejemplo 1 0 1 0 10.

#	Caso de prueba	opcode	Señales esperadas	Señales obtenidas (consola)	¿OK?
1	**Tipo R** *Aritmética entre registros (add, sub, mul, div)*	000000	1 0 1 0 10	1 0 1 0 10	Sí
2	**Tipo I: MOVI** *Mueve un inmediato a un registro*	001000	0 1 1 0 00	0 1 1 0 00	Sí
3	**Tipo I: otra instrucción** *ADDI, SUBI u otra que hayan definido*	001001	0 1 1 0 00	0 1 1 0 00	Sí
4	**Tipo J** *Salto incondicional (JMP / JUMP)*	000010	0 0 0 1 00	0 0 0 1 00	Sí
5	**Opcode no definido** *Debe quedar «seguro»: sin escribir registros ni saltar*	111111	0 0 0 0 00	0 0 0 0 00	Sí
6	**Secuencia R → J → I** *Las señales siguen al opcode actual, sin arrastrar valores anteriores*	000000 → 000010 → 001000	1 0 1 0 10 → 0 0 0 1 00 → 0 1 1 0 00	1 0 1 0 10 → 0 0 0 1 00 → 0 1 1 0 00	Sí

Ejercicio 2 — ALU

1. Código Verilog de la ALU

Dos operandos de 32 bits, un control de 4 bits, resultado de 32 bits y bandera de cero. Implementa suma, resta, multiplicación y división. Comenta la codificación del control que eligieron y qué hace tu ALU con un control no definido.

Tu código — la ALU

```
`timescale 1ns/1ns

module ALU (
    input wire [31:0] src1,
    input wire [31:0] src2,
    input wire [3:0] alu_control,
    output reg [31:0] result,
    output wire zero
);

    // Operaciones combinacionales con operandos sin signo.
    always @(*) begin
        result = 32'd0;

        case (alu_control)

            // MOVI: copiar el inmediato.
            4'b0000: result = src2;

            // MOV: copiar el primer operando.
            4'b0001: result = src1;

            // ADD.
            4'b0010: result = src1 + src2;

            // SUB.
            4'b0110: result = src1 - src2;

            // MULT: conservar los 32 bits inferiores.
            4'b1000: result = src1 * src2;

            // DIV: cociente entero sin signo.
            4'b1001: begin
                if (src2 == 32'd0)
                    result = 32'd0;
                else
                    result = src1 / src2;
            end

            // Control no definido.
            default: result = 32'd0;
```

```
        endcase
    end

    // Bandera activa cuando el resultado es cero.
    assign zero = (result == 32'd0);

endmodule
```

2. Testbench de la ALU

Prueba distintas combinaciones de operandos para cada operación (incluye negativos, cero y desbordes) y muestra resultado y bandera de cero.

Tu testbench — la ALU

```
`timescale 1ns/1ns

module tb_ALU;

    reg [31:0] src1;
    reg [31:0] src2;
    reg [3:0] alu_control;

    wire [31:0] result;
    wire zero;

    integer pruebas;
    integer errores;

    ALU uut (
        .src1(src1),
        .src2(src2),
        .alu_control(alu_control),
        .result(result),
        .zero(zero)
    );

    // Aplicar operandos y verificar resultado y bandera.
    task probar;
        input [3:0] control;
        input [31:0] operando1;
        input [31:0] operando2;
        input [31:0] esperado;
        input cero_esperado;

        begin
            alu_control = control;
```

```
            src1 = operando1;
            src2 = operando2;
            #10;

            pruebas = pruebas + 1;

            // $signed cambia la presentacion decimal.
            // No cambia la division sin signo del modulo ALU.
            $display(
                "Prueba %0d: ctl=%b src1=%0d src2=%0d -> result=%0d (0x%08h)
zero=%b",
                pruebas, alu_control,
                $signed(src1), $signed(src2),
                $signed(result), result, zero
            );

            if ((result !== esperado) ||
                (zero !== cero_esperado)) begin

                errores = errores + 1;

                $display(
                    "ERROR: esperado=%0d (0x%08h), zero=%b",
                    $signed(esperado), esperado, cero_esperado
                );
            end
        end
    endtask

    initial begin
        $dumpfile("ALU.vcd");
        $dumpvars(0, tb_ALU);

        src1 = 0;
        src2 = 0;
        alu_control = 0;
        pruebas = 0;
        errores = 0;

        $display("=== Pruebas de ALU ===");
        $display("Division SIN signo; division entre cero devuelve 0.");

        // 1. Suma: 10 + 20 = 30.
        probar(4'b0010, 32'd10, 32'd20, 32'd30, 1'b0);

        // 2. Resta positiva: 50 - 15 = 35.
        probar(4'b0110, 32'd50, 32'd15, 32'd35, 1'b0);
```

```
        // 3. Resta con resultado cero.
        probar(4'b0110, 32'd25, 32'd25, 32'd0, 1'b1);

        // 4. Resta negativa: 5 - 10 = 0xFFFFFFFB.
        // Interpretado con signo, representa -5.
        probar(4'b0110, 32'd5, 32'd10, 32'hFFFFFFFB, 1'b0);

        // 5. Multiplicacion: 6 * 7 = 42.
        probar(4'b1000, 32'd6, 32'd7, 32'd42, 1'b0);

        // 6. Multiplicacion con overflow.
        // 65536 * 65536 = 2^32; los 32 bits inferiores son 0.
        probar(4'b1000, 32'd65536, 32'd65536, 32'd0, 1'b1);

        // 7. Division exacta: 20 / 4 = 5.
        probar(4'b1001, 32'd20, 32'd4, 32'd5, 1'b0);

        // 8. Division entera con residuo: 7 / 2 = 3.
        probar(4'b1001, 32'd7, 32'd2, 32'd3, 1'b0);

        // 9. Division entre cero: decision del diseno.
        probar(4'b1001, 32'd20, 32'd0, 32'd0, 1'b1);

        // 10. Suma con overflow: 0xFFFFFFFF + 1 = 0.
        probar(4'b0010, 32'hFFFFFFFF, 32'd1, 32'd0, 1'b1);

        // 11. OperandOS en cero: 0 + 0 = 0.
        probar(4'b0010, 32'd0, 32'd0, 32'd0, 1'b1);

        // 12. Patron de bits de -20: 0xFFFFFFEC.
        // La ALU lo interpreta SIN signo: 4294967276.
        // 4294967276 / 4 = 1073741819.
        probar(
            4'b1001, 32'hFFFFFFEC, 32'd4,
            32'd1073741819, 1'b0
        );

        // 13. Control no definido: devuelve 0.
        probar(4'b1111, 32'd10, 32'd20, 32'd0, 1'b1);

        // 14. MOVI: copiar src2.
        probar(4'b0000, 32'd99, 32'd123, 32'd123, 1'b0);

        // 15. MOV: copiar src1.
        probar(4'b0001, 32'd45, 32'd99, 32'd45, 1'b0);

        $display(
            "=== Fin: pruebas=%0d, errores=%0d ===",
```

```
            pruebas, errores
        );

        $finish;
    end

endmodule
```

3. Captura de la consola

Debe verse el comando de compilación y ejecución, y una línea por prueba con control, operandos, resultado y bandera de cero.

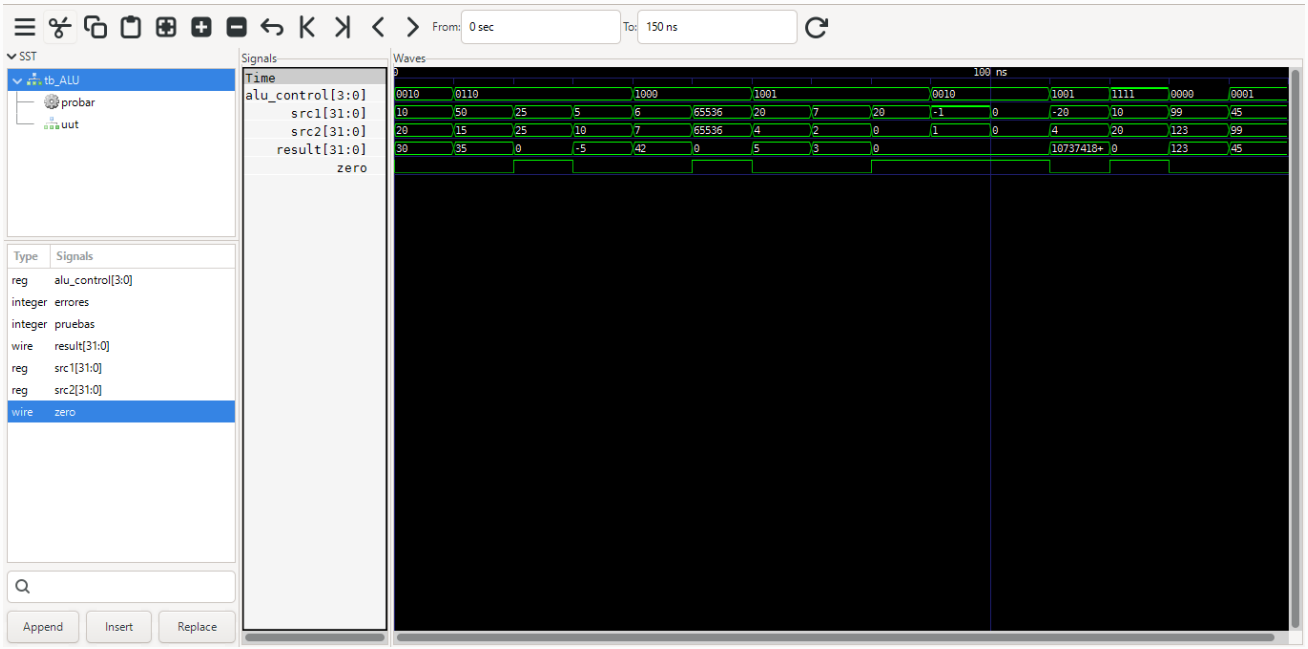
Captura — consola (ALU)

```
PS C:\Users\Usuario\Control_Registros> iverilog -s tb_ALU -o ALU_sim ALU.v tb_ALU.v
PS C:\Users\Usuario\Control_Registros> vvp ALU_sim | Tee-Object -FilePath ALU_consola.txt
VCD info: dumpfile ALU.vcd opened for output.
=== Pruebas de ALU ===
Division SIN signo; division entre cero devuelve 0.
Prueba 1: ctl=0010 src1=10 src2=20 -> result=30 (0x0000001e) zero=0
Prueba 2: ctl=0110 src1=50 src2=15 -> result=35 (0x00000023) zero=0
Prueba 3: ctl=0110 src1=25 src2=25 -> result=0 (0x00000000) zero=1
Prueba 4: ctl=0110 src1=5 src2=10 -> result=-5 (0xfffffffb) zero=0
Prueba 5: ctl=1000 src1=6 src2=7 -> result=42 (0x0000002a) zero=0
Prueba 6: ctl=1000 src1=65536 src2=65536 -> result=0 (0x00000000) zero=1
Prueba 7: ctl=1001 src1=20 src2=4 -> result=5 (0x00000005) zero=0
Prueba 8: ctl=1001 src1=7 src2=2 -> result=3 (0x00000003) zero=0
Prueba 9: ctl=1001 src1=20 src2=0 -> result=0 (0x00000000) zero=1
Prueba 10: ctl=0010 src1=-1 src2=1 -> result=0 (0x00000000) zero=1
Prueba 11: ctl=0010 src1=0 src2=0 -> result=0 (0x00000000) zero=1
Prueba 12: ctl=1001 src1=-20 src2=4 -> result=1073741819 (0x3ffffffb) zero=0
Prueba 13: ctl=1111 src1=10 src2=20 -> result=0 (0x00000000) zero=1
Prueba 14: ctl=0000 src1=99 src2=123 -> result=123 (0x0000007b) zero=0
Prueba 15: ctl=0001 src1=45 src2=99 -> result=45 (0x0000002d) zero=0
=== Fin: pruebas=15, errores=0 ===
tb_ALU.v:131: $finish called at 150 (1ns)
```

4. Captura de GTKWave

Control, los dos operandos, resultado y bandera de cero. Pon operandos y resultado en decimal con signo (clic derecho → Data Format → Signed Decimal) o en hexadecimal, y deja todas las pruebas a la vista.

Captura — GTKWave (ALU)



5. Detalle de los casos de prueba

Codificación del control de tu ALU

El control lo codifica cada grupo. Anota cuántos bits usó cada operación (4 bits) y qué hace tu ALU con un control no definido.

La codificación mantiene la compatibilidad con el módulo ALUControl de la práctica anterior. La división se realiza sin signo y entrega el cociente entero. Si el divisor es cero o el control no está definido, la ALU devuelve cero y activa la bandera zero. Los resultados se limitan a 32 bits, descartando los bits superiores cuando existe desbordamiento. El testbench utiliza $signed() para mostrar los operandos y resultados en decimal con signo; esto no cambia la interpretación sin signo de la división.

Operación	Control (4 bits)
Suma	0010
Resta	0110
Multiplicación	1000
División	1001
Control no definido	1111 como ejemplo; devuelve result = 0 y zero = 1
Operación adicional: MOVI	0000 — copia src2
Operación adicional: MOV	0001 — copia src1

Casos de prueba

Elige tus operandos para cumplir cada descripción, calcula el resultado y la bandera de cero a mano antes de simular, y después copia lo que dice la consola.

#	Caso de prueba	Entradas			Esperado (a mano)		Obtenido (consola)		OK
		Control	Operando 1	Operando 2	Resultado	Cero	Resultado	Cero	
1	**Suma** *Dos positivos pequeños*	0010	10	20	30	0	30	0	Sí
2	**Resta positiva** *Operando 1 > operando 2*	0110	50	15	35	0	35	0	Sí
3	**Resta con resultado 0** *Operando 1 = operando 2: la bandera Cero debe subir*	0110	25	25	0	1	0	1	Sí
4	**Resta negativa** *Operando 1 < operando 2: complemento a 2*	0110	5	10	-5	0	-5	0	Sí
5	**Multiplicación** *Dos positivos pequeños*	1000	6	7	42	0	42	0	Sí
6	**Multiplicación con overflow** *El producto no cabe en 32 bits: se trunca (ej. 65536 × 65536)*	1000	65536	65536	0	1	0	1	Sí
7	**División exacta** *Sin residuo*	1001	20	4	5	0	5	0	Sí
8	**División con residuo** *El resultado se trunca (ej. 7 / 2)*	1001	7	2	3	0	3	0	Sí
9	**División entre cero** *¿Qué devuelve tu ALU? Documéntalo*	1001	20	0	0	1	0	1	Sí
10	**Suma con overflow** *0xFFFFFFFF + 1: el resultado da la vuelta a 0*	0010	0xFFFFFFFF	1	0	1	0	1	Sí
11	**Operandos en cero** *0 + 0: Cero = 1*	0010	0	0	0	1	0	1	Sí
12	**División con negativo** *Operando 1 negativo (ej. -20 / 4): ¿con signo o sin signo?*	1001	−20 (0xFFFFFFEC)	4	1073741819	0	1073741819	0	Sí
13	**Control no definido**	1111	10	20	0	1	0	1	Sí
14	**MOVI**	0000	99	123	123	0	123	0	Sí
15	**MOV**	0001	45	99	45	0	45	0	Sí

Punto de control final — marca cada casilla

- ✓ Unidad de Control y ALU comentadas, con todos los puertos del enunciado.
- ✓ Un testbench por módulo, comentado, con $dumpfile("nombre.vcd") y $dumpvars(0, nombre_del_tb).
- ✓ Captura de consola de cada testbench, con los comandos visibles.
- ✓ Captura de GTKWave de cada testbench, con las señales legibles y todas las pruebas a la vista.

- ✓ Tablas de casos completas: esperado calculado a mano antes de simular, obtenido copiado de la consola.
- ✓ Codificación del control de la ALU documentada, y decisiones tomadas (división entre cero, signo, control no definido).

Guarda tus módulos y testbenches: son parte del procesador MIPS que van a construir.